DEPARTMENT OF
NETWORKED SYSTEMS
AND SERVICES

Biztonsági tesztelés

VIHIAC07 – IT biztonság, 2026

Dr. Futóné Dr. Papp Dorottya

CrySyS Lab, BME

dpapp@crysys.hu

www.crysys.hu

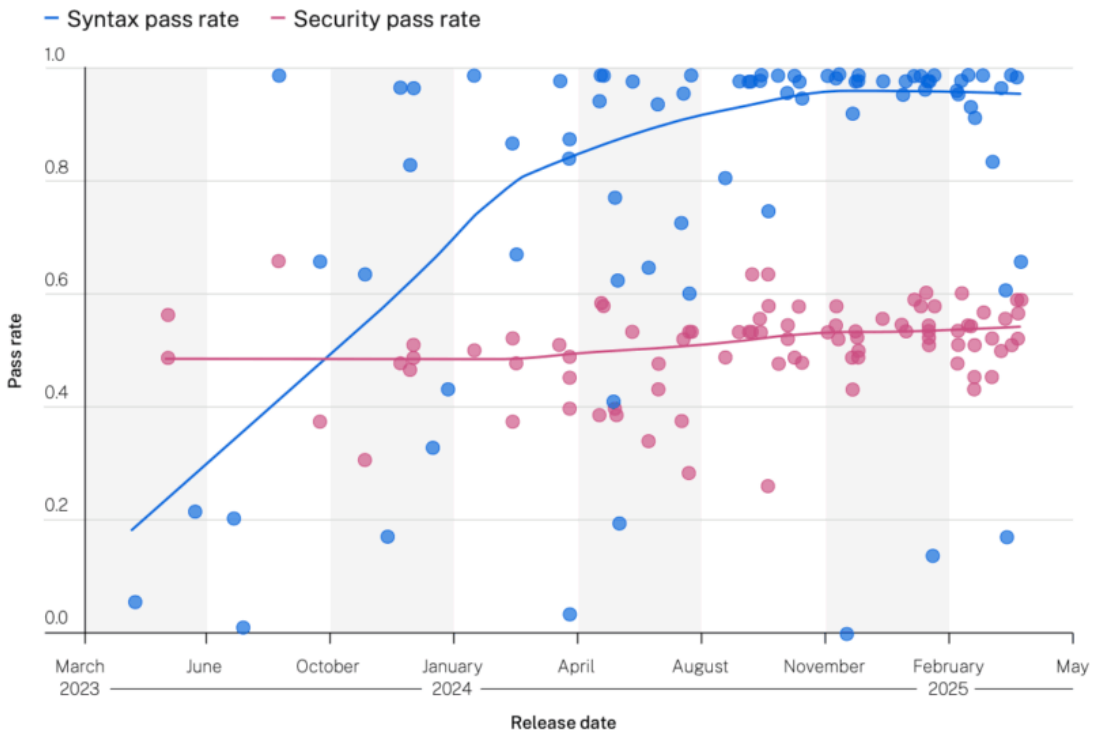
MŰEGYETEM 1782

Motiváció

- A tesztelés és a minőségbiztosítás mindig is fontos volt
- Az LLM-ek forradalmasították a szoftverfejlesztést
- De nem feltétlenül jó értelemben

FIGURE 1

Security and Syntax Pass Rates vs LLM Release Date



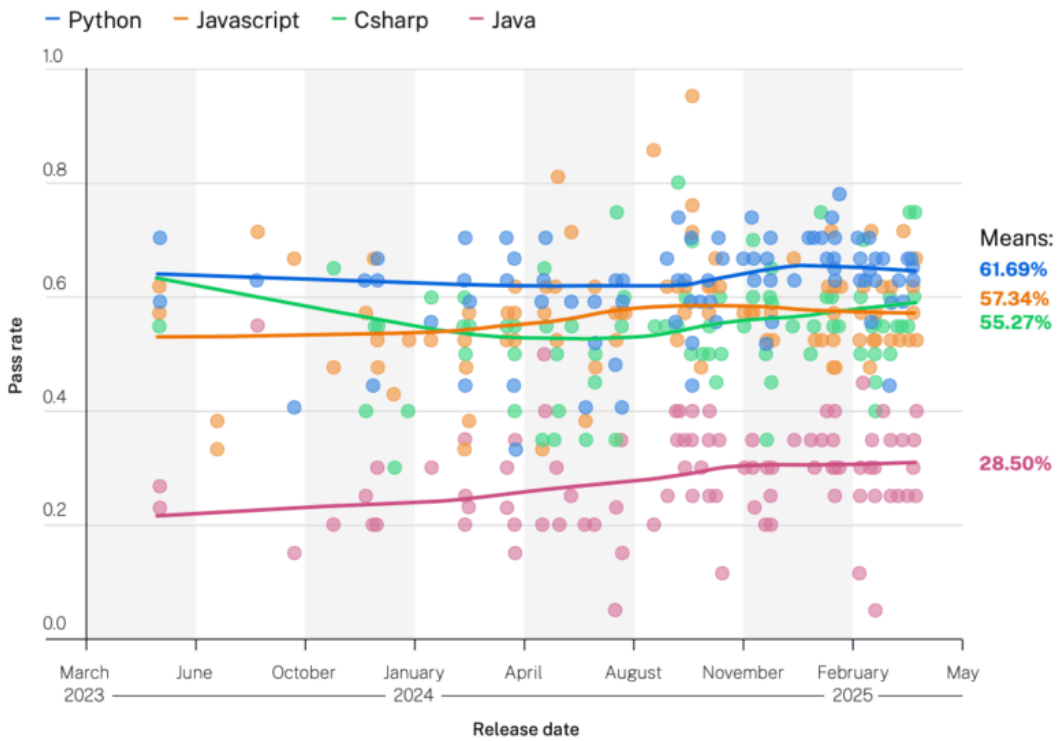
Note: Security rate only shown for dates/groups with syntax pass rate > 50%

Motiváció

- A tesztelés és a minőségbiztosítás mindig is fontos volt
- Az LLM-ek forradalmasították a szoftverfejlesztést
- De nem feltétlenül jó értelemben

FIGURE 2

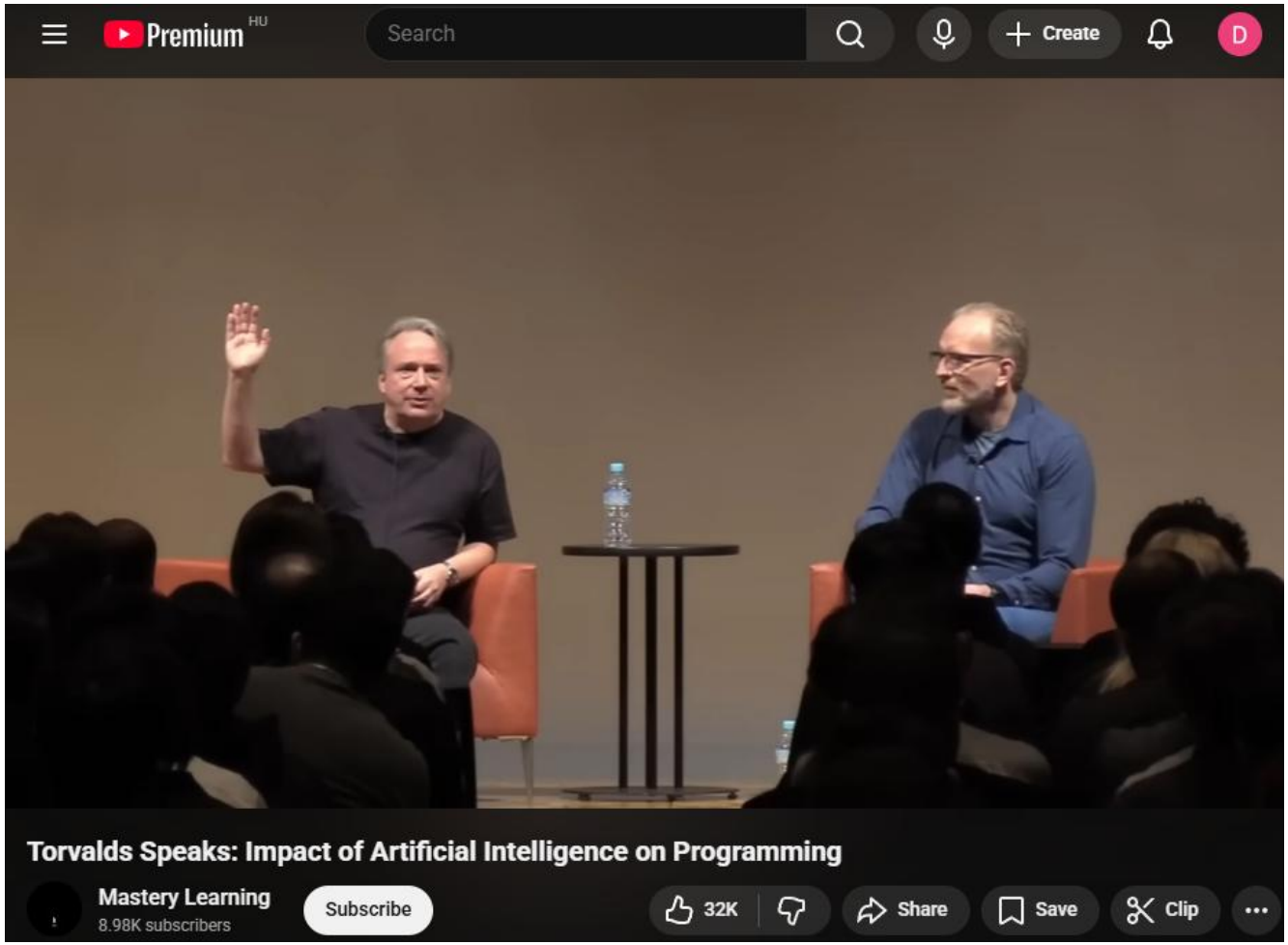
Security Pass Rate vs LLM Release Date, Stratified by Language



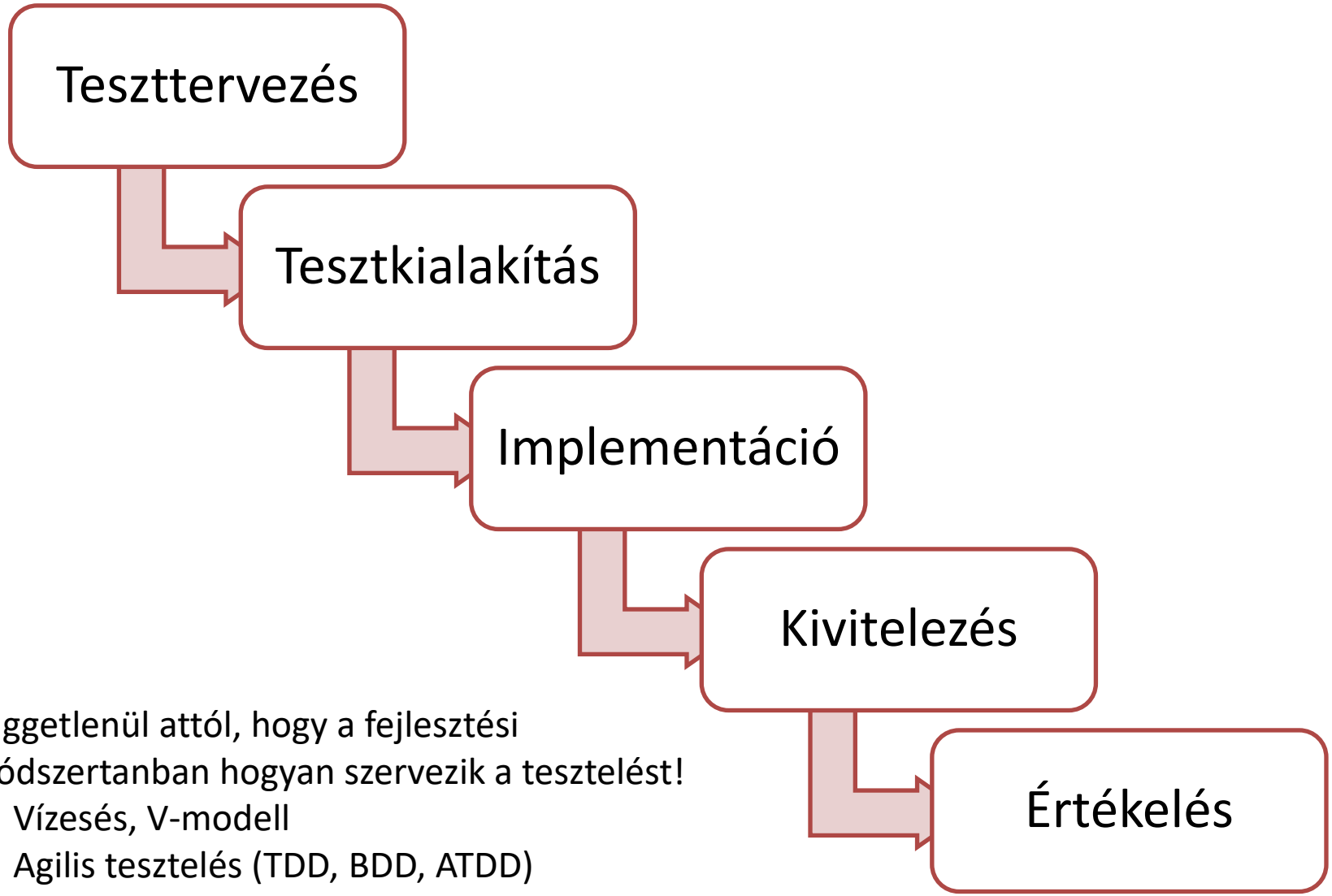
Note: Security rate only shown for dates/groups with syntax pass rate > 50%

Motiváció

- https://www.youtube.com/watch?v=VHHT6W-N0ak (2:04)



A szoftvertesztelés általános folyamata



Függetlenül attól, hogy a fejlesztési módszertanban hogyan szervezik a tesztelést!

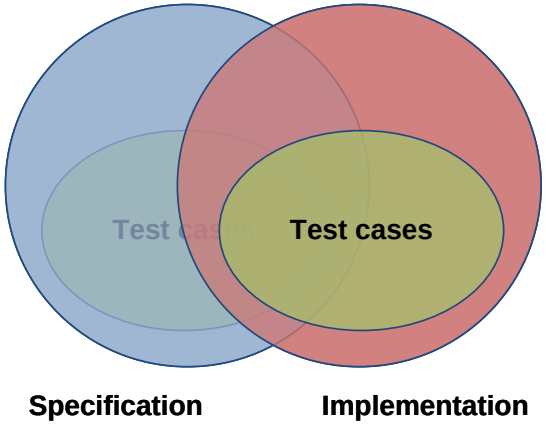
- Vízesés, V-modell
- Agilis tesztelés (TDD, BDD, ATDD)

Általános tesztelési megközelítések

- **Manuális** vagy **automatizált**
- **Formális** vagy **gyakorlati** megközelítés
- **Belső** tesztelés vagy külső **pentest**

- Statikus vagy dinamikus
  - **Statikus**: terveketen vagy forráskódon
  - **Dinamikus**: futási időben

- Black-box / white-box
  - Black-box: **specifikáció** alapján
  - White-box: **implementáció** alapján
  - Gray-box: a kettő kombinációja



Általános tesztelés módszerek

- Funkcionális tesztelés
  - Unit tesztelés
  - Integrációs tesztelés
  - Rendszer tesztelés
  - Elfogadási tesztelés

Általános tesztelés módszerek

- Funkcionális tesztelés

- Nem-funkcionális tesztelés
  - Teljesítménytesztelés
  - Terheléses tesztelés
  - Failover tesztelés
  - Helyreállítás tesztelés

Általános tesztelés módszerek

- Functional testing
- Non-functional testing
- **Biztonsági tesztelés**: annak ellenőrzése, hogy a biztonsági követelmények teljesülnek-e

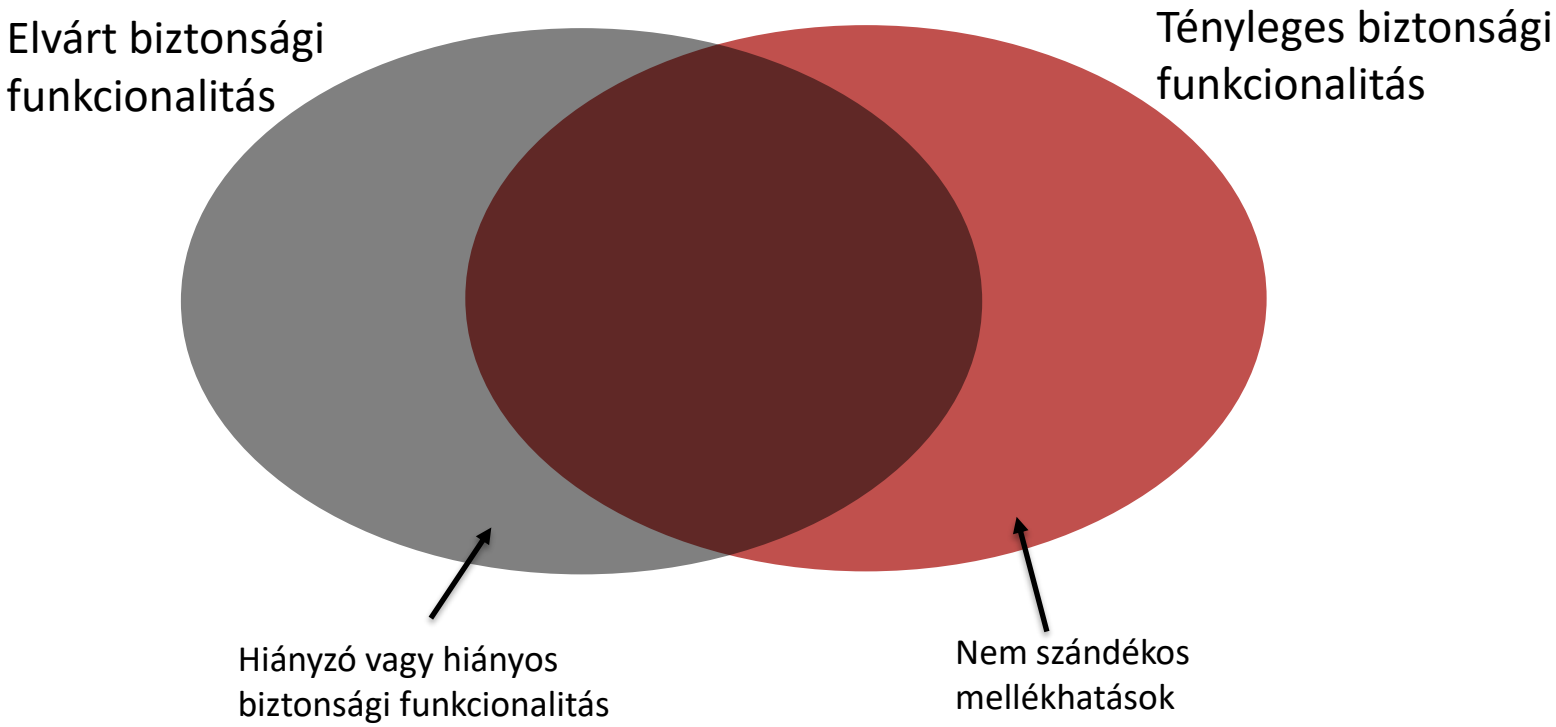
Biztonsági tesztelés

- „Program testing can be a very effective way to show the presence of bugs, but it is hopelessly inadequate for showing their absence.” – Edsger W. Dijkstra

- Folyamatos újraprioritizálás!
  - Végtelen számú lehetséges módja van annak, hogy egy alkalmazás meghibásodjon
  - Korlátozott tesztelési idő és erőforrások

A biztonsági tesztelés célja

- A sebezhetőségek azonosítása, a biztonsági funkciók biztosítása
  - Pozitív követelmények: ami az elvárt
  - Negatív követelmények: ami elfogadhatatlan



A biztonsági tesztelés kihívásai

- A megfelelő gondolkodásmód kialakítása
  - „A dobozon kívül” gondolkodás
  - Támadóként való gondolkozás
  - Az automatizált eszközök nem gondolkodnak kreatívan

A biztonsági tesztelés kihívásai

- A megfelelő gondolkodásmód kialakítása
- A tesztelt elem megértése
  - Az alkalmazás pontos dokumentálását igényli
  - Javítja az összes teszt hatékonyságát

A biztonsági tesztelés kihívásai

- Developing the right mindset
- Understanding the subject
- Jó mérőszámok kidolgozása
  - Több oktatásra/képzésre van szükség?
  - Egy bizonyos biztonsági mechanizmus nem teljesen érthető?
  - A biztonsággal kapcsolatos problémák összesített száma csökken?

Az előadás főbb pontjai

- Tervek áttekintése
- Komponens-szintű biztonsági tesztelés
  - Statikus program analízis technikák a biztonsági tesztelésben
  - Dinamikus program analízis technikák a biztonsági tesztelésben
- A biztonsági tesztelés integrálása CI/CD pipeline-okba
- Élő rendszerek biztonsági tesztelése

A tervek áttekintése

Design review - introduction

- A tervezés során felmerülő biztonsági problémák azonosítása
  - A biztonsági követelmények meghatározása és felülvizsgálata
    - » Hiányzó vagy hiányos követelmények
  - A magas szintű rendszertervezés és architektúra felülvizsgálata
    - » A tervezési hibák minél korábbi feltárása
  - A tervezésen alapuló megvalósítás során felmerülő potenciális problémák
    - » Különösen fontos biztonsági szempontból kritikus komponensek („hotspotok”) – például a hitelesítési modul

- Dokumentum alapú
  - Követelmények specifikációja
  - Architektúrális áttekintő dokumentum
  - Tervezési dokumentumok

Sérülékenység keresés komponensekben

Áttekintés

- Program analízis technikákon alapul

- Statikus program analízis technikák
  - Code review
  - Taint analízis
  - Szimbolikus végrehajtás

- Dinamikus program analízis technikák
  - Debuggolás és trace generálás
  - Fuzz tesztelés
  - Dinamikus taint analízis

Statikus technikák

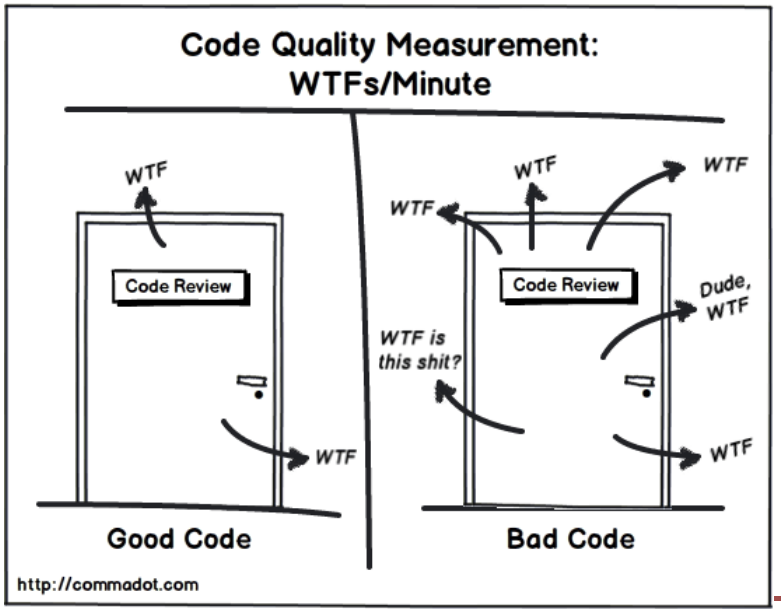
- Bemenet: forráskód, közbenső nyelv, gépi kód
- Az utasításokat csak értelmezzük, nem hajtjuk végre
- Lehet manuális és/vagy automatizált

- Előnyök
  - Skálázható
  - Az SDL korai szakaszaiban alkalmazható

- Hátrányok
  - Unalmas manuális folyamat → eszközök automatizálhatják a folyamatot
  - Az eszközök csak előre meghatározott sebezhetőségi mintákat keresnek
  - Nincs futási információ → hamis pozitív eredmények

Code Review

- A kód manuális áttekintése
  - Az emberi szakértő sorról sorra elolvassa az utasításokat, hogy azonosítsa a sebezhető pontokat
  - Támadó model szükséges hozzá (de legalábbis a támadási felület)
  - Előny: emberi kreativitás.
  - Hátrány: unalmas folyamat, sok területről kell szakértelem (alkalmazásarchitektúra, implementációs technika, biztonság stb.)



Code Review

- A kód manuális áttekintése

- Mit kell keresni? → CWE adatbázis
  - Veszélyes konstrukciók (pl. függvények)
  - Injection lehetőségek (SQL, shell végrehajtás, XML, …)
  - Integer kezelés, túlcsordulás, signed/unsigned hibák, csonkolás
  - Hiányzó input validáció (támadási felület), hiányzó kimeneti escape-elés (HTML, XML, logs, …)
  - Nyelv-specifikus problémák

Code Review

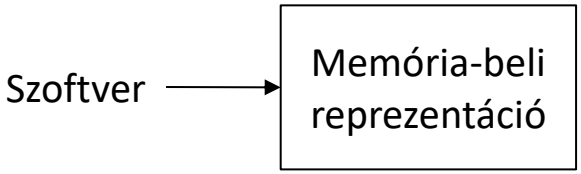
- A kód manuális áttekintése

- Static application security testing (SAST) → skálázhatóság
  - Automatizált code review
  - A SAST eszköz elemzi az utasításokat és jelentést készít a potenciális problémákról, az embernek csak a jelentést kell áttekintenie
  - Előny: nincs szükség szakértelemre
  - Hátrány: az eszköz csak azt jelenti, amit a beépített mintái alapján keresnie kell

SAST eszközök

- Bemenet: forráskód, bináris kód
- A sebezhetőségek felismerése az utasítások „elolvasásával”

- Static application security testing aka automatizált code review



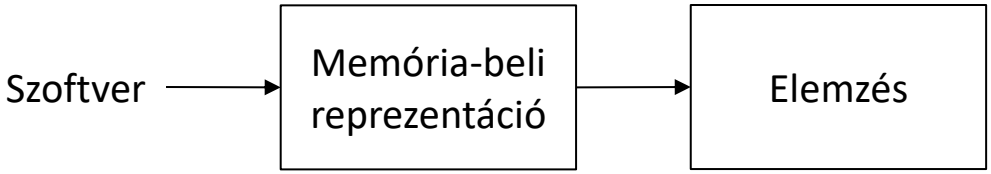
  – A bemenetet átalakítjuk egy memória-beli reprezentációra

  – Példa: absztrakt szintaxis fa, köztes reprezentáció

SAST tools

- Input: source code, binary code
- Detect vulnerabilities by „reading” the instructions

- Static application security testing aka automated code review



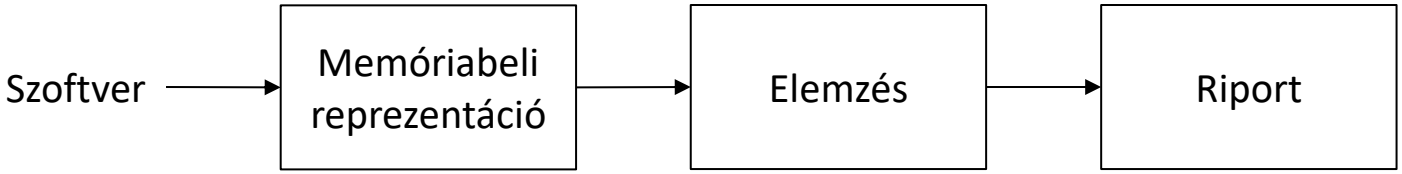
  – Az eszköz elvégzi az implementált elemzéseket

  – Példák: vezérlési folyam elemzés, dependencia elemzés, típus inferencia és típus ellenőrzés, program szeletés

SAST tools

- Input: source code, binary code
- Detect vulnerabilities by „reading” the instructions

- Static application security testing aka automated code review



  – Riportban: hiba/figyelmeztés, szoftverben hol?

  – Az eredményeket validálni kell!
    » Hamis pozitív: interakció zárt komponensekkel/külső szolgáltatásokkal
    » Hamis negatív: az eszköz csak azt találja meg, amit meg tud találni!

Mit keres a SAST?

- Mintaalapú megközelítés
  - Kifejezetten veszélyes konstrukciók keresése

Mit keres a SAST?

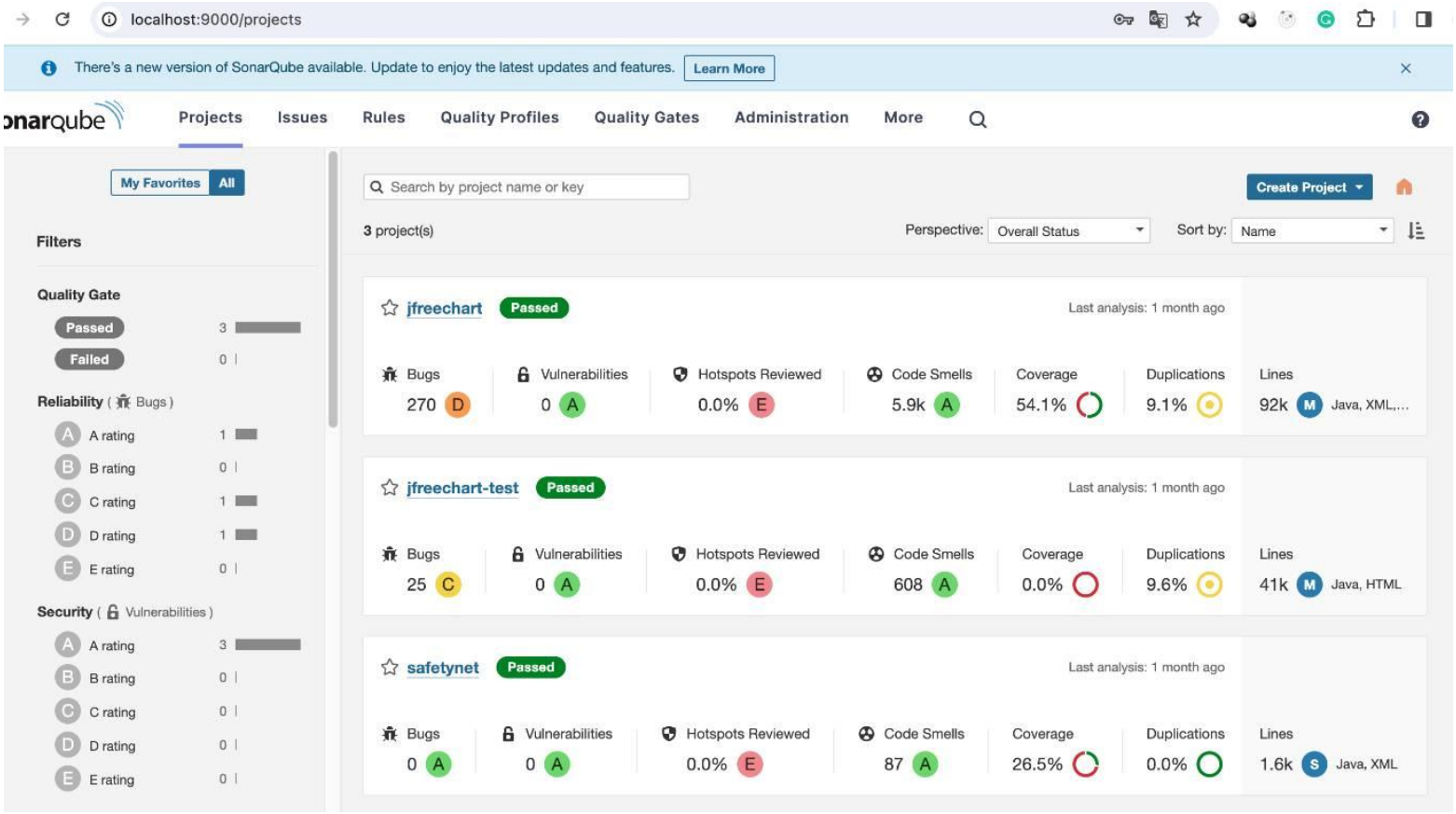
- Mintaalapú megközelítés
- Általános "red flag" heurisztikák
  - Injection lehetőségek (SQL, shell execution, XML, …)
  - Integer kezelés, túlcsordulás, signed/unsigned hibák, csonkolás
  - Hiányzó input validáció (támadási felület), hiányzó kimeneti escape-elés (HTML, XML, logs, …)
  - …

Mit keres a SAST?

- Mintaalapú megközelítés
- Általános "red flag" heurisztikák
- Konkrét nyelv- vagy technológiafüggő minták
  - C: fix méretű pufferek, pufferkezelés és -túlcsordulás
  - Java/C#: szerializáció és deszerializáció
  - Java/C#: üres catch blokkok, széleskörű kivételkezelés
  - …

SAST tools

- Átfogó lista: https://owasp.org/www-community/Source_Code_Analysis_Tools



Dinamikus analízis

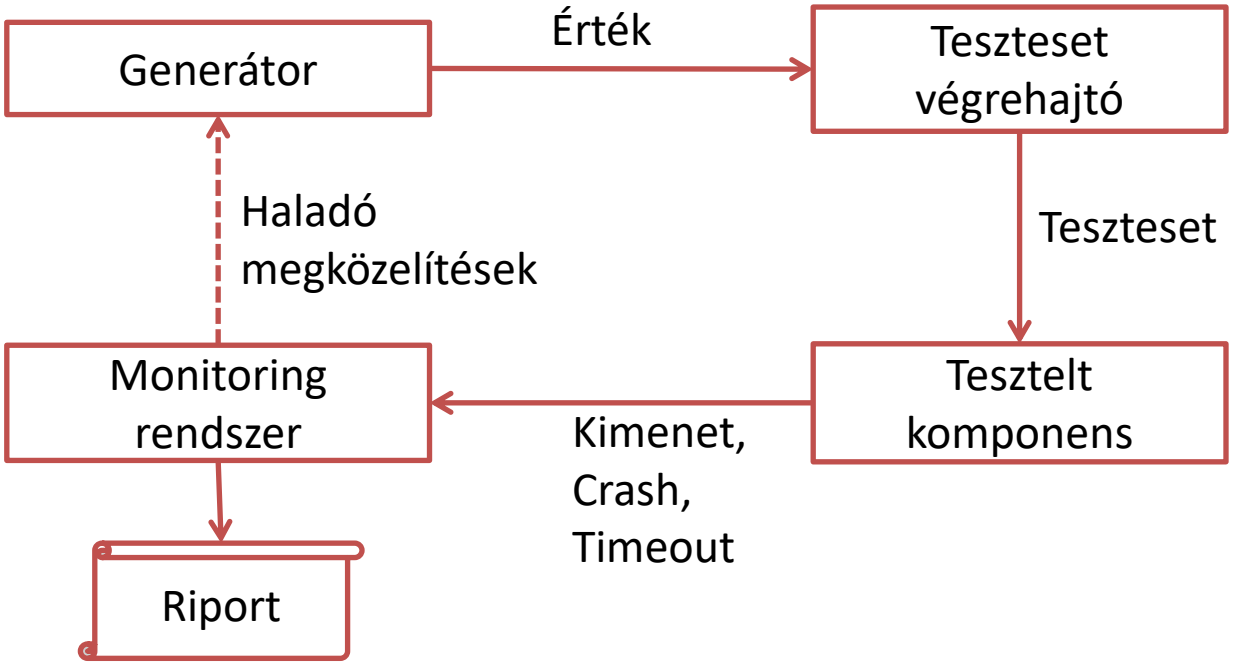
- A szoftvert valós körülmények között, futás közben elemzi

- Előnyök
  - Vannak futási idejű információk → pontosabb eredmények
  - A jelentett sebezhetőségek biztosan a szoftverben vannak

- Hátrányok
  - Programinstrumentációra lehet szükség (nem triviális fordított kódhoz)
  - Alacsony kódlefedettség
  - Csak a megfigyelt viselkedésről lehet következtetéseket levonni

Fuzzing

- A véletlenszerű bemeneteket automatikusan generáljuk
- A tesztelés során figyeljük a hibákat/hibás viselkedést



- Kihívás: elérni a „mély” kódot

Fuzzing

- Példák tipikus hibákra, amelyek megtalálhatóak
  - Puffer túlcsordulás
    → Szekvenciális közelítés
    (bináris keresés)
  - Hiányzó input validáció
  - Memória korrupció (C/C++)
  - Integer problémák: bemenetként 0, -1, minimum és maximum integer értékek, 2-hatványok, …
  - …

Manuális vs. automatizált DAST

Futási idejű analízis	Manuális	Automatizált
Módszer	Saját tesztesetek egyedi fejlesztése	Tesztvektorok szisztematikus és automatizált generálása
Előny	Speciális problémák szakértelem alapján (mélyreható)	A tipikus hibák többségének megtalálása (szélességben)
Tesztek száma	Tucatnyi	Több tízezer
Erőfeszítés	Egyedi tesztkód minden egyes tesztesethez	Az eszközök adaptálása és összekapcsolása egyszer, a tesztek többszöri futtatása
Lefedettség	Fókusz a legkockázatosabb részeken	A lefedettség jelentős növekedése

- Legjobb kombinálni
- Az automatizált tesztek segíthetik a kézi értékelést
  - A tesztelési terület bővítése
  - A „kritikus pontok” (problémás részek) azonosítása

DAST tools

- Példák: https://owasp.org/www-community/Vulnerability_Scanning_Tools

```
                    american fuzzy lop 1.86b (test)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 2 sec        │  cycles done : 0      │
│   last new path : none seen yet                      │  total paths : 1      │
│ last uniq crash : 0 days, 0 hrs, 0 min, 2 sec        │ uniq crashes : 1      │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 0 (0.00%)         │    map density : 2 (0.00%)             │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.00 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 1 (100.00%)            │
│ stage execs : 1464/5000 (29.28%)    │  new edges on : 1 (100.00%)            │
│ total execs : 1697                  │ total crashes : 39 (1 unique)          │
│  exec speed : 626.5/sec             │   total hangs : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 0/16, 1/15, 0/13                      │    levels : 1          │
│  byte flips : 0/2, 0/1, 0/0                         │   pending : 1          │
│ arithmetics : 0/112, 0/25, 0/0                      │  pend fav : 1          │
│  known ints : 0/10, 0/28, 0/0                       │ own finds : 0          │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 0/0, 0/0                              │  variable : 0          │
│        trim : n/a, 0.00%                            ├────────────────────────┘
└─────────────────────────────────────────────────────┘             [cpu: 92%]
```

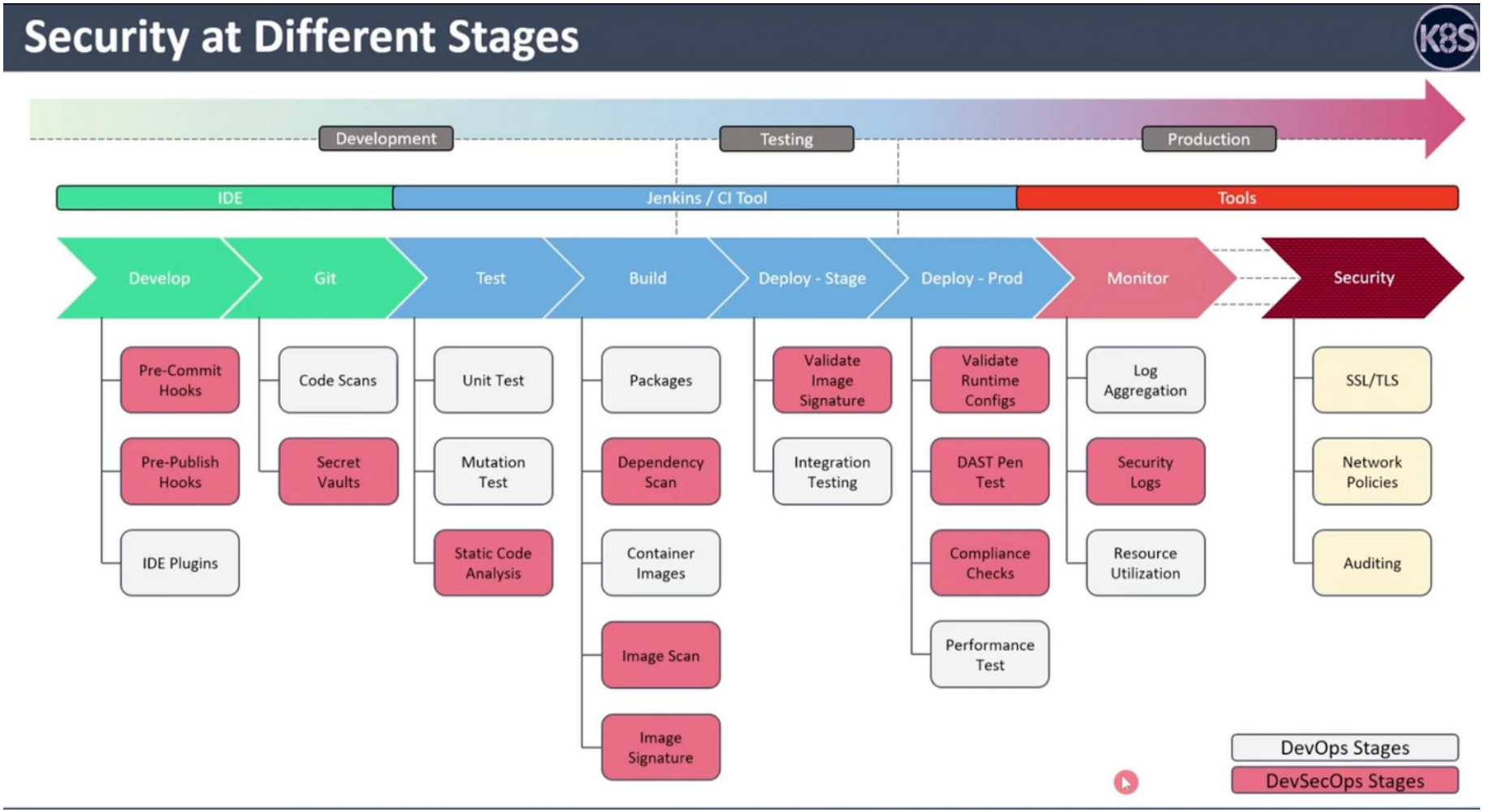
SAST vs DAST

- SAST előnyei:
  - Nagy lefedettség
  - Skálázhatóság
  - Korán elkezdhető
- SAST hátrányai:
  - Téves riasztások
  - Túlterhelő riportok
- DAST előnyei:
  - Pontosság
  - Tényleges sebezhetőségeket jelent
- DAST hátrányai:
  - Instrumentáció
  - Alacsony lefedettség
  - Csak a megfigyelt viselkedésről tud bármit mondani

Konklúzió: kombináljuk a kettőt!

A biztonsági tesztelés integrációja

Biztonsági tesztelés a CI/CD pipeline-ban



https://github.com/Taiwolawal/CICD-DEVSECOPS

Függőségi vizsgálat

- Aka software composition analysis (SCA)

- A függőségi fa átvizsgálása ismert sebezhetőségek után
  - CVE adatbázis
  - Software Bill of Materials (SBOM)

SPDX®

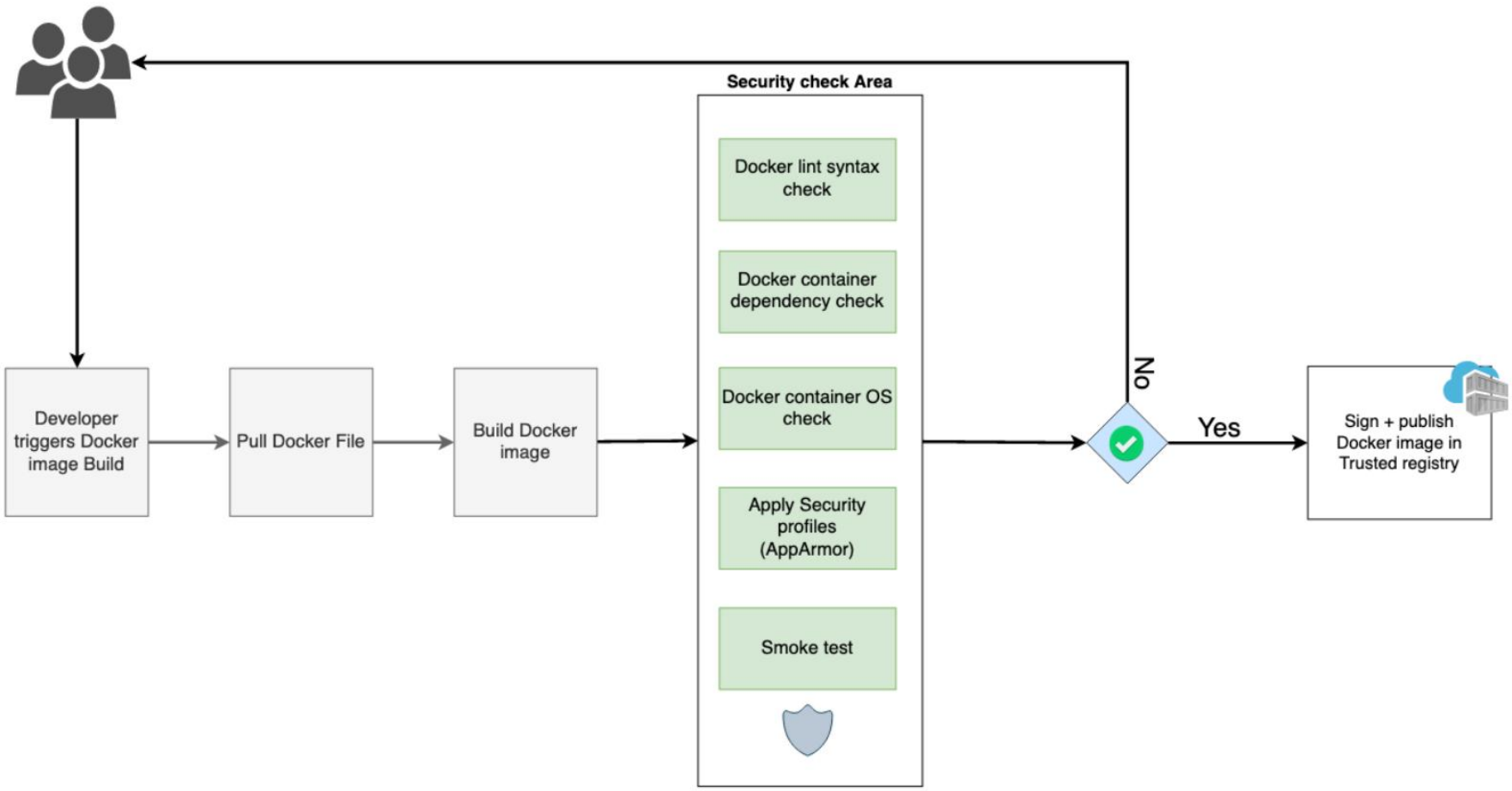
CycloneDX

- Releváns eszközök: https://owasp.org/www-community/Component_Analysis

Konténer image-ek szkennelése

- Nem-biztonságos konténerek felismerése
  - Elavult könyvtárak
  - Helytelen konfigurációk
  - Elavult operációs rendszerek
- Megfelelőségi ellenőrzés

Konténer image-ek szkennelése



https://owasp.org/www-project-devsecops-guideline/latest/02f-Container-Vulnerability-Scanning

Élő rendszerek biztonsági tesztelése

Áttekintés

- Cél: a rendszerben még mindig fennálló sebezhetőségek felkutatása

- Bug bounty programok
- Penetrációs tesztelés
- Red teaming

Penetrációs tesztelés

- Bemenet: futó rendszer teszt- vagy production környezetben
- A beállítás hasonló egy tényleges támadáshoz
  → (majdnem) Production alkalmazás/rendszer előnyösebb!
  → Szükség lehet statikus és dinamikus technikákra egyaránt

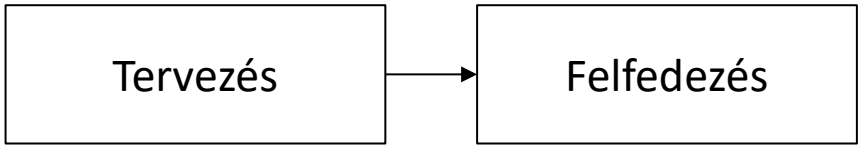
Penetrációs tesztelés

- Tervezés: teszt fontos mellékfeltételei és
  - Mely komponensek részei a scope-nak? Valódi vagy dummy rendszer?
  - Időzítéssel kapcsolatos szempontok?
- Nincs tényleges tesztelés

Tervezés

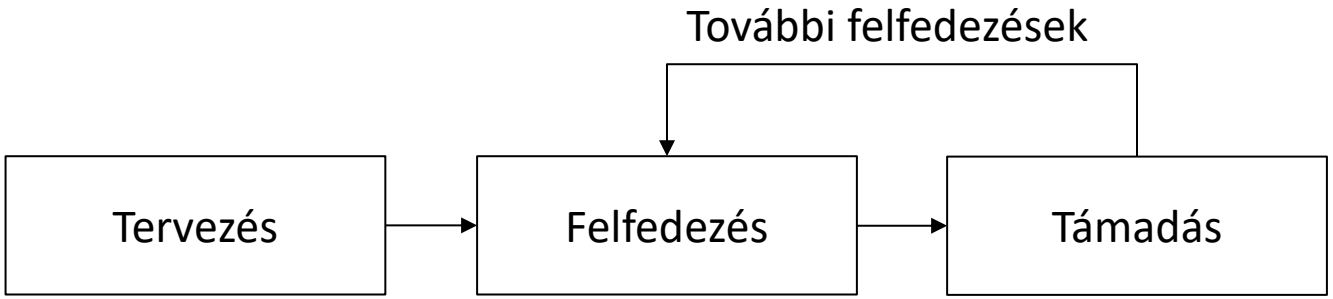
Penetrációs tesztelés

- Felfedezés: az összes elérhető interfész felfedezése és felsorolása, majd sebezhetőségi elemzés
  - Nyilvános sebezhetőségi adatbázisok (pl. CVE) ismert komponensekhez
  - Sérülékenység szkennerek



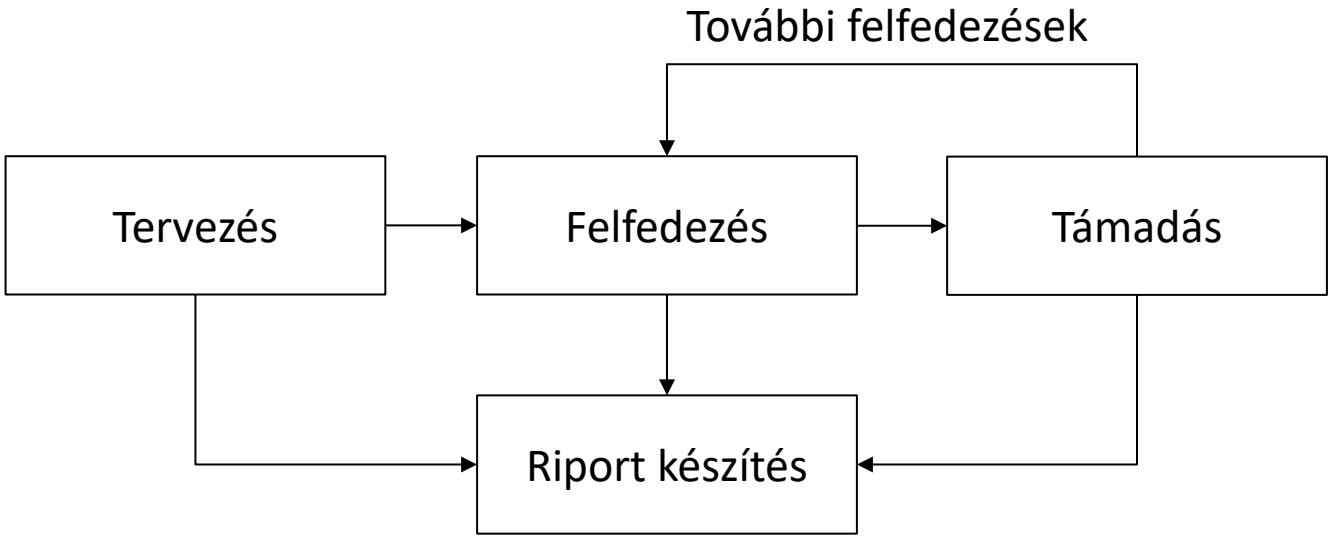
Penetrációs tesztelés

- Támadás: azonosított interfészek tesztelése támadásokkal
  - Sikeres kihasználás
    → további információ API-król és szolgáltatásokról, új komponensek felfedezése
    → támadási felület növelése



Penetrációs tesztelés

- Riport készítés: a többi fázissal párhuzamosan, dokumentálni kell az összes eredményt és azok becsült súlyosságát



Összefoglaló

Tanulságok

- A biztonsági tesztelés ortogonális a hagyományos tesztelési módszerekre
- Cél: sérülékenységek azonosítása
- A fejlesztési életciklus minden szakaszára vannak technikák

- **→ Többféle, integrált technika kiegyensúlyozott kombinációja**

IT security education program (BSc, MSc)

more info: http://www.crysys.hu/education/

Ellenőrző kérdések

Ellenőrző kérdések

- Mi a biztonsági tesztelés célja?
- Melyek a biztonsági tesztelés különböző megközelítései?
- Mi a különbség a manuális és az automatizált tesztelés között?
- Mi a különbség a statikus és a dinamikus tesztelés között?
- Mi a különbség a white box és a black box tesztelés között?
- Melyek a tesztelési folyamat lépései?
- Melyek a biztonsági tesztelés tipikus kihívásai?
- Melyek a manuális kódellenőrzés jellemzői?
- Melyek az automatizált biztonsági tesztelés jellemzői?
- Mi a célja a tervek felülvizsgálatának?
- Hogyan történik a code review (mind manuálisan, mind automatizáltan)?
- Melyek a statikus elemzés előnyei és hátrányai?

Ellenőrző kérdések

- Melyek a dinamikus elemzés előnyei és hátrányai?
- Mi a fuzzing fő elve és általános munkafolyamata?
- Milyen biztonsági tesztelési technikák alkalmazhatók egy CI/CD folyamatban?
- Mi a behatolási tesztelés célja?
- Melyek a behatolási tesztelés fő fázisai?

Referenciák

References

- Cybok: Software Security
  - https://www.cybok.org/media/downloads/Software_Security_v1.0.1.pdf
- Microsoft: Security Development Lifecycle
  - https://www.microsoft.com/en-us/SDL/process/requirements.aspx
- OWASP Web Security Testing Guide
  - https://owasp.org/www-project-web-security-testing-guide